INFORME DE LA GUÍA DE APRENDIZAJE N° 5

MANEJO DE ARCHIVOS DE TEXTO, ARCHIVOS BINARIOS Y EXCEPCIONES EN JAVA

Curso: Taller de Lenguajes de Programación
Estudiante: Marcos Riquelme Huahuiza Sarayasi
Institución: Instituto Superior Tecnológico de La Joya

1. Introducción

En esta sesión trabajé con almacenamiento permanente en Java. Hasta ahora, los programas que hacíamos perdían toda la información al cerrarse. Con esta guía aprendí a guardar datos en archivos del computador para que se queden ahí aunque el programa termine. El proyecto base sigue siendo GameStore Escolar, y el módulo de esta vez fue Auditoría, Historial y Respaldos.

2. Objetivo

Implementar programas que guarden y lean información en archivos de texto plano y en archivos binarios, controlando los errores de lectura y escritura con excepciones (IOException).

3. Herramientas utilizadas

- JDK (OpenJDK 17 o superior), que incluye el compilador javac y la máquina virtual de Java.
- Visual Studio Code con la extensión Extension Pack for Java.
- Terminal de PowerShell integrada en VS Code.
- Carpeta de trabajo: D:\Taller_de_Lenguaje_de_Programacion\Sesion_5\codigo

4. Marco teórico resumido

Archivos de texto. Guardan la información como caracteres que una persona puede leer, por ejemplo un .txt. En Java se usan clases como FileWriter, PrintWriter y BufferedReader.

Archivos binarios. Guardan los datos directamente como bytes, por ejemplo objetos serializados. Son útiles para respaldos o estructuras más complejas. Se usan clases como FileOutputStream y ObjectOutputStream.

Excepciones. Trabajar con archivos puede fallar por cosas que el programa no controla: que el archivo no exista, que el disco esté lleno o que falten permisos. Java lo avisa con una IOException. Como es una excepción "checked", el compilador obliga a capturarla con try-catch o a declararla con throws.

Try-with-resources. Es una forma de escribir el try que cierra el archivo automáticamente al terminar, incluso si ocurre un error. Así no se queda ningún archivo abierto por descuido.

5. Desarrollo

5.1 Trabajo 1: registro en texto plano (HistorialLog.java)

Esta clase guarda en historial_canjes.txt cada canje que hace un estudiante en la tienda. El archivo se abre en modo append (el segundo parámetro true de FileWriter), por lo que cada

ejecución agrega una línea nueva sin borrar las anteriores. También tiene un método para leer el historial línea por línea con BufferedReader.

```
import java.io.FileWriter;
import java.io.PrintWriter;
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class HistorialLog {

    // Trabajo 1: guardar un registro en un archivo de texto (modo append)
    public static void registrarCanje(String estudiante, String
itemCanjeado) {
        // true = append: agrega al final sin borrar lo anterior
        // try-with-resources: cierra el archivo solo, incluso si hay error
        try (PrintWriter pw = new PrintWriter(new
FileWriter("historial_canjes.txt", true))) {
            pw.println("Estudiante: " + estudiante + " | Ítem: " +
itemCanjeado);
            System.out.println("Registro de canje guardado exitosamente en
el archivo de texto.");
        } catch (IOException e) {
            System.out.println("Error de E/S al escribir en el archivo de
texto: " + e.getMessage());
        }
    }

    // Leer el historial de texto línea por línea
    public static void leerHistorial() {
        try (BufferedReader br = new BufferedReader(new
FileReader("historial_canjes.txt"))) {
            String linea;
            System.out.println("--- HISTORIAL DE CANJES (TEXT-FILE) ---");
            while ((linea = br.readLine()) != null) {
                System.out.println(linea);
            }
        } catch (IOException e) {
            System.out.println("No se encontró el archivo o hubo un error
al leerlo: " + e.getMessage());
        }
    }
}
```

5.2 Trabajo 2: respaldo binario (RespaldoBinario.java)

Esta clase guarda la configuración del sistema en el archivo binario config_respaldo.dat usando ObjectOutputStream, y luego la recupera con ObjectInputStream. Al leer, hay que convertir el objeto de vuelta a String, y por eso también se captura ClassNotFoundException.

```
import java.io.FileOutputStream;
import java.io.ObjectOutputStream;
import java.io.FileInputStream;
import java.io.ObjectInputStream;
import java.io.IOException;

public class RespaldoBinario {

    // Trabajo 2: guardar la configuración en un archivo binario (.dat)
    public static void guardarRespaldoConfig(String configuracion) {
        try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("config_respaldo.dat"))) {
            oos.writeObject(configuracion);
            System.out.println("Respaldo binario generado correctamente.");
        } catch (IOException e) {
            System.out.println("Error al escribir el archivo binario: " +
e.getMessage());
        }
    }

    // Leer el respaldo binario
    public static void leerRespaldoConfig() {
        try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("config_respaldo.dat"))) {
            String configLeida = (String) ois.readObject();
            System.out.println("Configuración recuperada desde el binario:
" + configLeida);
        } catch (IOException | ClassNotFoundException e) {
            System.out.println("Error al leer el archivo binario: " +
e.getMessage());
        }
    }
}
```

5.3 Clase de ejecución (PrincipalArchivos.java)

Esta clase tiene el método main y sirve para probar los dos módulos anteriores. Al inicio incluí un comentario que explica por qué es obligatorio usar try-catch o throws con archivos.

```java
/*
 * ¿Por qué es obligatorio usar try-catch (o throws) con archivos?
 * Porque las operaciones de E/S dependen de factores externos al programa
 * (archivo inexistente, disco lleno, sin permisos). Java los representa con
 * IOException, una excepción "checked": el compilador EXIGE que la captures
 * (try-catch) o la declares (throws IOException). Así el programa no se cae
 * y puede avisar del error de forma controlada.
 */
public class PrincipalArchivos {
    public static void main(String[] args) {
        // Probando Text-File (Trabajo 1)
        HistorialLog.registrarCanje("Ana Torres", "Skin Legendario");
        HistorialLog.leerHistorial();

        // Probando Archivo Binario (Trabajo 2)
        RespaldoBinario.guardarRespaldoConfig("ModoMantto=false;Servidor=Activo");
        RespaldoBinario.leerRespaldoConfig();
    }
}
```

6. Compilación y ejecución

Los pasos que seguí en la terminal de VS Code, dentro de la carpeta codigo, fueron:

1. Compilar todas las clases: `javac *.java`
2. Ejecutar la clase principal: `java PrincipalArchivos`

Dificultad encontrada

Al principio escribí `java *.java` y después `java PrincipalArchivos` sin haber compilado, y en ambos casos me salió el error `Could not find or load main class`. Entendí que java solo sirve para ejecutar una clase ya compilada (con el nombre de la clase, sin .java), y que antes hay que compilar con javac, que es el que genera los archivos .class. Después de compilar, el programa ya se pudo ejecutar.

Resultado esperado

```
Registro de canje guardado exitosamente en el archivo de texto.
--- HISTORIAL DE CANJES (TEXT-FILE) ---
Estudiante: Ana Torres | Ítem: Skin Legendario
Respaldo binario generado correctamente.
Configuración recuperada desde el binario:
ModoMantto=false;Servidor=Activo
```

Además, en la carpeta del proyecto aparecieron dos archivos nuevos creados por el propio código:

- `historial_canjes.txt`: texto plano, se puede abrir con cualquier editor.
- `config_respaldo.dat`: archivo binario, no está pensado para leerse como texto.

Si el programa se ejecuta varias veces, historial_canjes.txt acumula una línea nueva por cada ejecución, porque se abre en modo append. En cambio, config_respaldo.dat se sobrescribe y queda igual.

Captura del resultado

Imagen de la terminal con la salida del programa:

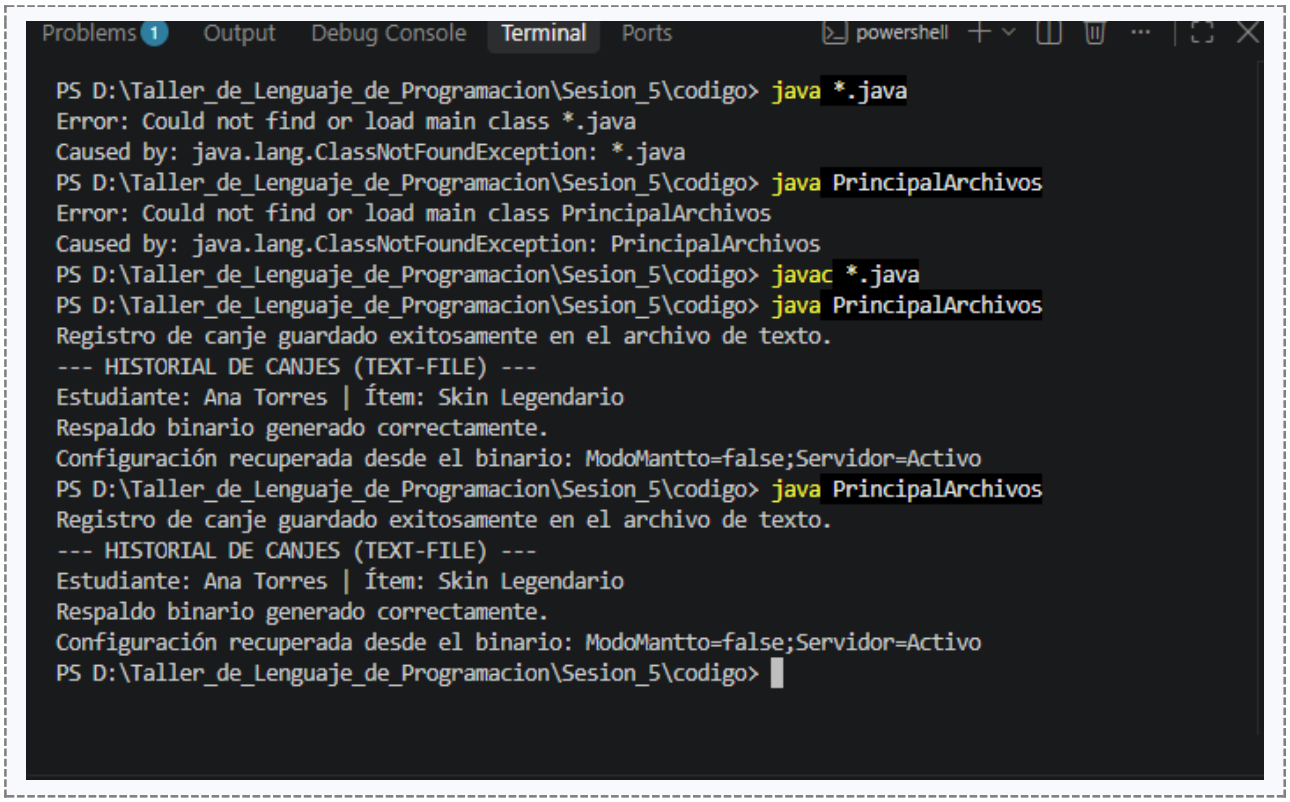
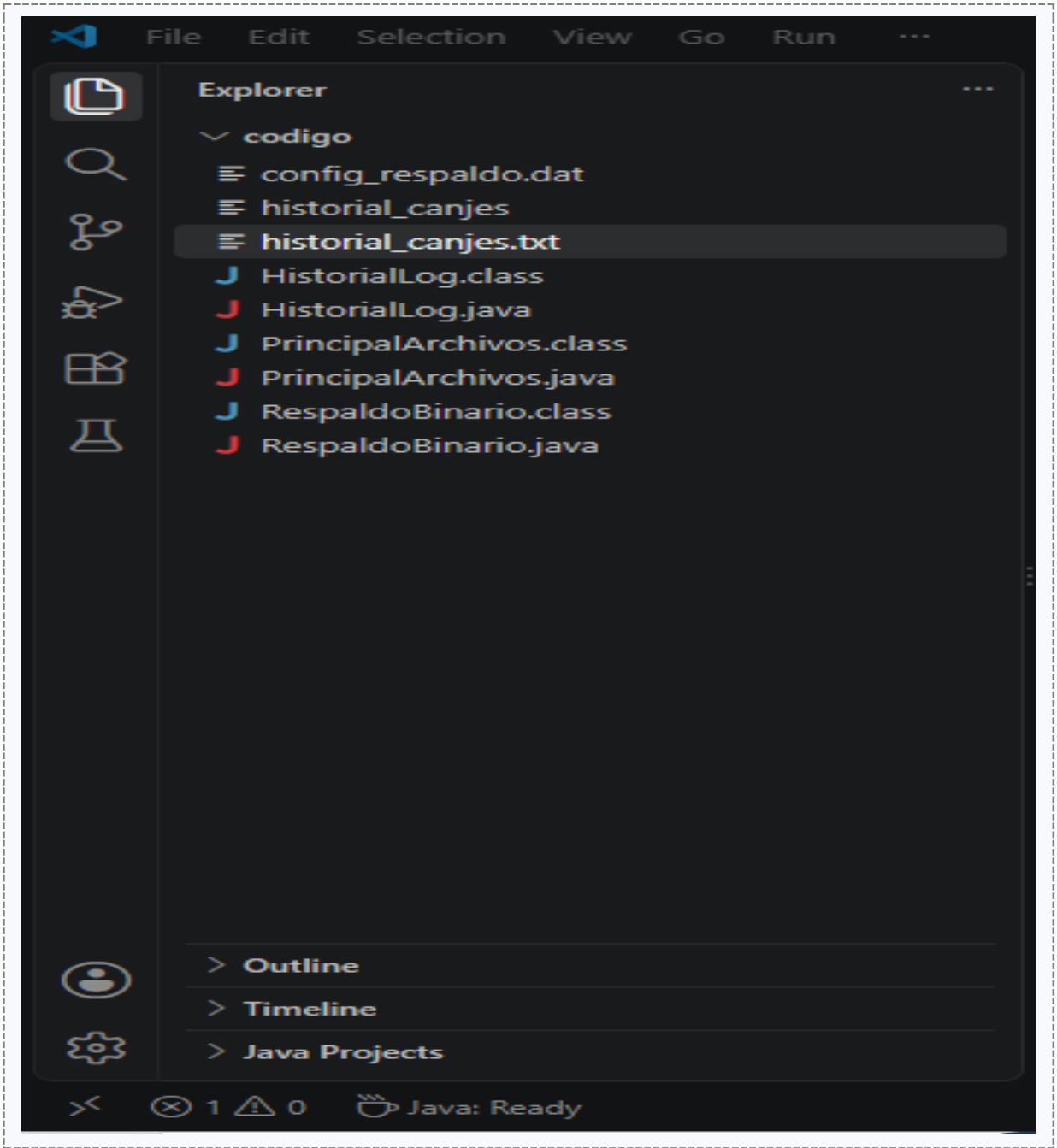


Imagen de la carpeta con los archivos historial_canjes.txt y config_respaldo.dat creados:



7. Sustentación conceptual

Es obligatorio usar try-catch (o throws) al trabajar con archivos porque estas operaciones dependen de cosas externas al programa, como que el archivo exista, que haya espacio en el disco o que se tengan permisos. Java representa esos fallos con IOException, que es una excepción "checked", así que el compilador no deja compilar si no se maneja. Con try-catch el programa no se cae de golpe y puede mostrar un mensaje claro del problema.

También usé try-with-resources en todos los métodos para que los archivos se cierren solos, lo cual es parte del orden en la gestión de recursos que pide la guía.

8. Conclusiones

- Aprendí a guardar y leer información en archivos de texto y en archivos binarios usando las clases de Java I/O.
- Entendí la diferencia entre ambos tipos: el texto se puede leer a simple vista y el binario guarda los datos en bytes.
- Comprendí que el manejo de excepciones no es opcional con archivos, y que try-with-resources evita dejar archivos abiertos.
- Aprendí la diferencia entre compilar (javac) y ejecutar (java), que fue el error que más me costó al comienzo.
- El modo append es importante cuando se quiere llevar un historial sin borrar los registros anteriores.